ECE 385

Spring 2025

Final Project Report

XP Liu

Introduction

This project explores the design and implementation of a custom GPU pipeline on an FPGA for real-time 3D graphics rendering. The system supports general triangle-based models, allowing for the visualization of more complex objects composed of arbitrary triangle meshes. The goal is to demonstrate how software and hardware components can be integrated to efficiently perform 3D rendering tasks such as transformation, rasterization, and depth testing.

The architecture is divided between a software front end running on a MicroBlaze soft-core processor and a hardware back end implemented using custom logic on the FPGA. The processor handles keyboard input, performs 3D transformations using quaternions, and prepares interpolated scanline data for each triangle. These scanlines are streamed to the hardware GPU over an AXI4-Lite interface.

The GPU module receives each scanline and performs pixel-level interpolation, Z-buffer depth comparison, and conditional writes to a frame buffer. This frame buffer is read by a separate display controller that outputs HDMI video. A palette-based color system allows efficient memory usage, while dual-port BRAM enables concurrent access from the GPU and display pipeline. Double buffering is used to eliminate tearing and ensure smooth visual output.

Through this project, I gain hands-on experience with 3D graphics principles—including vertex shading, triangle rasterization, and Z-buffering—while also developing practical skills in hardware/software co-design, AXI bus communication, and real-time embedded systems.

Written Description

This project implements a custom GPU pipeline on an FPGA that supports real-time rendering of arbitrary 3D models composed of triangles. The rendering process accepts any mesh defined by a collection of triangle primitives. The rendering process includes vertex transformation, triangle rasterization, texture mapping, and Z-buffer-based depth testing.

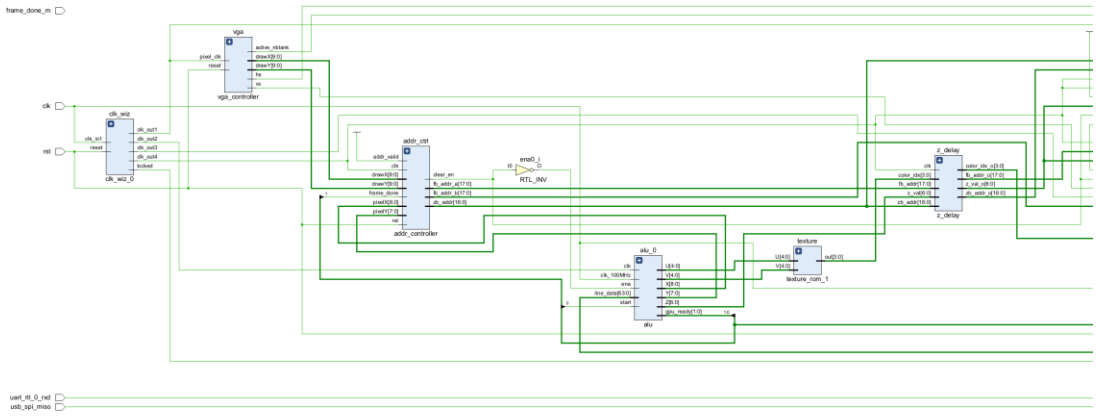
The software component, running on a MicroBlaze soft-core processor, is responsible for high-level control and 3D geometry processing. It accepts input from a USB keyboard, applies quaternion-based rotations to the object, and transforms model-space vertices into screen-space coordinates. For each triangle, the software generates interpolated scanline data containing pixel row (y), horizontal start and end positions (x0 to x1), and corresponding texture (u, v) and depth (z) values.

This scanline data is sent to a hardware GPU module via an AXI4-Lite interface. The GPU receives each scanline and performs per-pixel interpolation and depth comparison using a Z-buffer. Pixels that pass the depth test are written to a frame buffer using palette-indexed color values. Both the frame buffer and Z-buffer are implemented using dual-port BRAMs to support concurrent access from the GPU and the display logic.

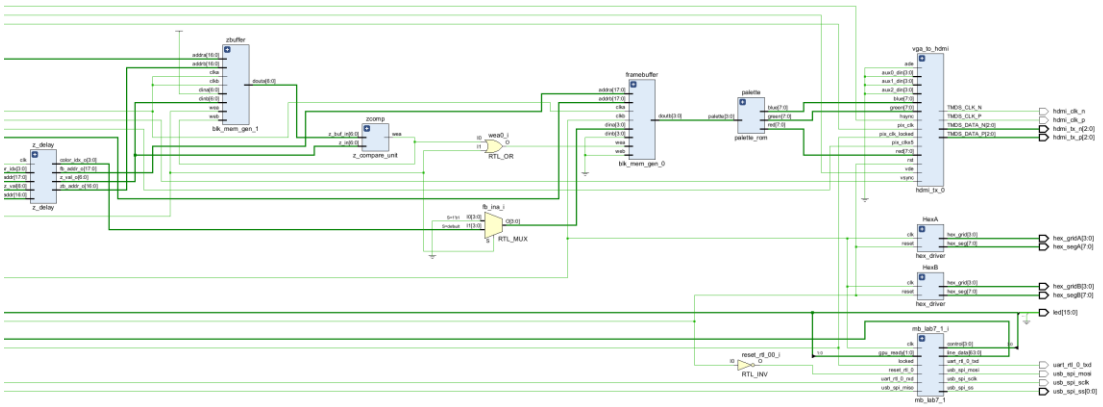
The system features double buffering to enable flicker-free display. While one buffer is being written by the GPU, the other is read by the HDMI controller. A dedicated VGA timing generator provides pixel coordinates to fetch display data, which is converted from a 4-bit palette index to 24-bit RGB using a palette ROM. The resulting pixel stream is then passed to a hardware HDMI encoder for output.

With support for general triangle meshes, this pipeline enables rendering of complex 3D scenes in real time. It serves as a foundational example of hardware-software co-design, fixed-function graphics processing, and memory-mapped GPU command streaming on an FPGA platform.

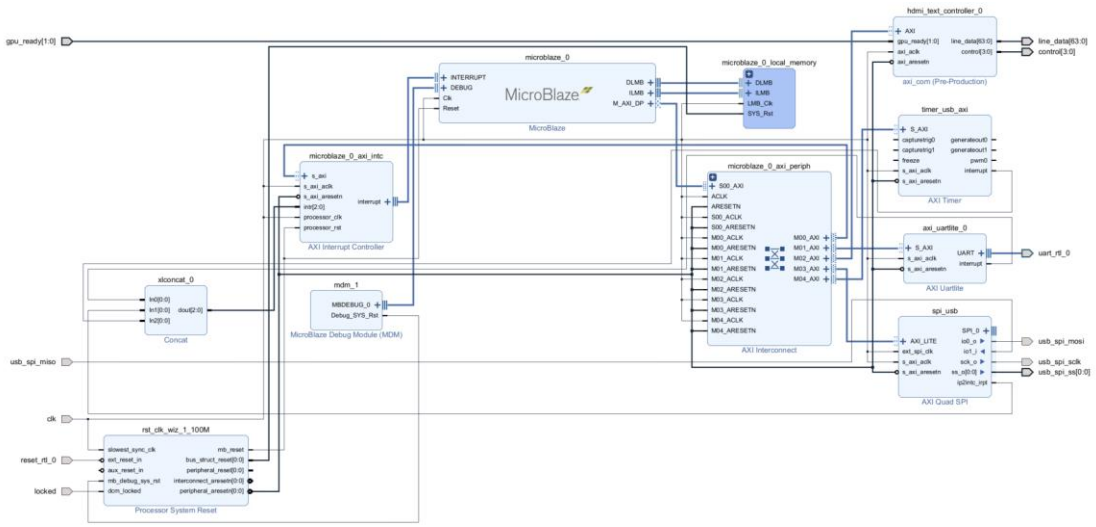
Block Diagrams



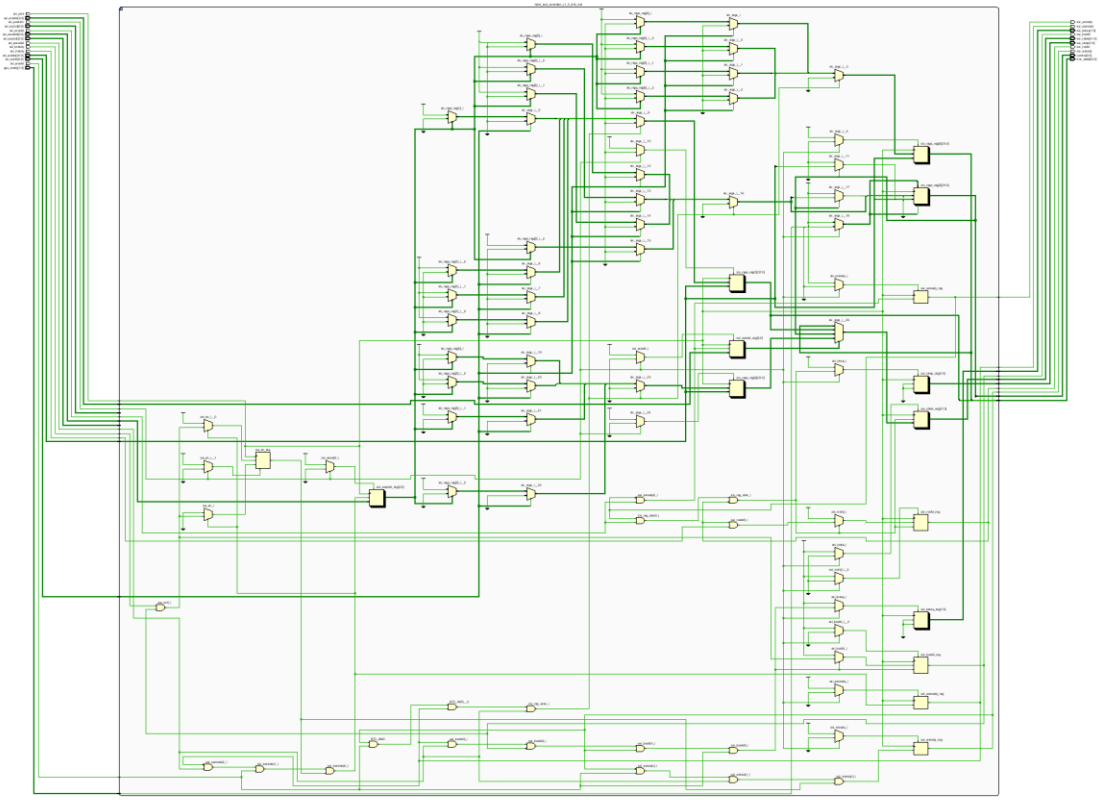
Top Level Block Diagram(Left)



Top Level Block Diagram(Right)



Block Design Diagram



AXI Com IP Block Diagram

Module Descriptions

Module: gpu_top

Inputs:

- System clock and reset:
  clk, rst
- UART interface:
  uart_rtl_0_rxd
- SPI interface (USB-to-SPI bridge):
  usb_spi_miso
- Frame control input:
  frame_done_m

Outputs:

- HDMI differential signals:
  hdmi_clk_p, hdmi_clk_n, hdmi_tx_p[2:0], hdmi_tx_n[2:0]
- UART and SPI outputs:
  uart_rtl_0_txd, usb_spi_mosi, usb_spi_sclk, usb_spi_ss[0]
- LED status output:
  led[15:0]
- Seven-segment display outputs:
  hex_segA, hex_gridA, hex_segB, hex_gridB

Description:

The gpu_top module is the complete top-level design for a hardware GPU pipeline implemented on FPGA with HDMI output. It integrates clock generation, AXI communication via a MicroBlaze soft processor, a GPU-style ALU for triangle rasterization, and dual memory buffers (frame buffer and z-buffer) for 3D graphics rendering with hidden surface removal. Textures are sampled using a texture_rom, and final pixels are colored via a programmable palette.

The module includes multiple subsystems:

- Clocking subsystem (clk_wiz) to generate necessary pixel and logic clocks.
- AXI system (mb_lab7_1) that receives geometry data and rendering control signals from software.
- GPU logic including alu, z_compare_unit, z_delay, and address generation (addr_controller).
- Dual-port BRAMs as frame buffer and z-buffer (framebuffer, zbuffer) to support

concurrent write/read by GPU and VGA controller.

- VGA to HDMI conversion pipeline using vga_controller and hdmi_tx_0.
- User I/O interfaces like SPI, UART, and 7-segment displays for interaction and debugging.

A status LED output displays internal states including GPU readiness, frame flags, and control signals, while the HDMI output continuously renders pixel data based on rasterized triangles with depth comparison and texturing.

Purpose:

This module demonstrates a full FPGA-based graphics system designed for education in digital systems and hardware/software co-design (e.g., in UIUC ECE 385). It emulates a simplified 3D graphics pipeline that performs rasterization, Z-buffering, and texture mapping in real-time. By integrating a soft processor, memory-mapped control, and hardware-accelerated rendering stages, it provides a hands-on platform to learn GPU principles, AXI interfacing, and video signal generation.

Module: alu

Inputs:

- clk: GPU processing clock
- clk_100MHz: Clock for loading data
- start: Triggers a new line interpolation
- ena: Enables interpolation step
- line_data[63:0]: Encoded triangle line data (positions, texture, depth)

Outputs:

- X[8:0], Y[7:0]: Pixel position
- Z[6:0]: Interpolated depth
- U[4:0], V[4:0]: Texture coordinates
- gpu_ready[1:0]: Status flags (01 = busy, 10 = done)

Description:

The alu module performs per-pixel linear interpolation across a scanline of a triangle. On receiving a start signal, it latches parameters (start/end X, Z, U, V, and fixed Y) from line_data. It then computes fixed-point deltas and steps through each X coordinate from x_start to x_end, outputting interpolated U, V, Z values. The module uses signed Q8.8 arithmetic to maintain precision and includes control logic to indicate GPU status during operation.

Purpose:

Acts as a core GPU rasterization unit, enabling pixel-wise calculation of texture coordinates and depth for a single triangle scanline.

Module: z_compare_unit

Inputs:

- z_buf_in[6:0]: Z-buffer value at the current pixel
- z_in[6:0]: Interpolated Z value from ALU

Output:

- wea: Write enable if z_in < z_buf_in

Description:

This module performs depth testing by comparing the current interpolated Z value to the Z-buffered value. It asserts the wea signal when the current pixel is closer to the camera (i.e., has a smaller Z), allowing subsequent pipeline stages to update the framebuffer and Z-buffer accordingly.

Purpose:

Implements per-pixel Z-buffer visibility logic to support hidden surface removal in 3D rendering.

Module: z_delay

Inputs:

- clk: Clock for registering values
- z_val[6:0]: Depth value to delay
- color_idx[3:0]: Corresponding color index
- fb_addr[17:0]: Frame buffer address
- zb_addr[16:0]: Z-buffer address

Outputs:

- z_val_o, color_idx_o, fb_addr_o, zb_addr_o: Delayed signals (2-cycle pipeline)

Description:

The z_delay module introduces a two-cycle delay for key pixel attributes (Z, color, addresses). It aligns interpolated data with memory access timing and helps maintain consistent pipeline behavior when storing to Z-buffer and framebuffer.

Purpose:

Enables pipelined synchronization between ALU output and memory write operations, ensuring correct timing and avoiding hazards.

Module: zbuffer (Z-Buffer BRAM)

Inputs:

- clka: Clock for Z-buffer clearing (GPU side)
- wea: Write enable for clearing (active during clear_en)
- addra[16:0]: Address for clearing
- dina[6:0]: Data to write during clearing (typically all 1s)
- clkb: Clock for GPU Z-write (same or faster clock)
- web: Write enable if a closer pixel is detected (from z_compare_unit)
- addrb[16:0]: Z-buffer address for current pixel
- dinb[6:0]: Interpolated Z-value to write
  Output:
- douta[6:0]: Z-value read during clearing
- doutb[6:0]: Z-value read before write comparison

Description:

This dual-port BRAM module acts as a Z-buffer, storing the depth of the last written pixel for each screen location. It is used for hidden surface removal in 3D rendering. During frame startup, the Z-buffer is cleared to a high value (e.g., 127) via port A. During rasterization, port B reads the current Z at each address and overwrites it only if a closer Z-value (lower number) is provided, as determined by z_compare_unit.

Purpose:

Maintains per-pixel depth information to determine visibility during triangle rendering. Prevents overdraw and enforces correct depth ordering in 3D scenes.

Module: framebuffer (Framebuffer BRAM)

Inputs:

- clka: GPU-side clock
- wea: Write enable (active during new visible pixel or clear_en)
- addra[17:0]: Address of pixel to write

- dina[3:0]: 4-bit color index (points to palette)
- clkb: VGA or HDMI pixel clock
- web: Constant 0 (no write on read port)
- addrb[17:0]: Address of pixel to read for display
  Output:
- doutb[3:0]: Color index for current display pixel

Description:

This dual-port BRAM module serves as the Framebuffer, holding the color index of every pixel on screen. The GPU writes color indices during triangle rasterization. Meanwhile, the display system (e.g., VGA controller) reads from the second port using current scan coordinates to retrieve pixel data for real-time display.

Double buffering is supported by offsetting read and write base addresses (fb_addr_a, fb_addr_b) to alternate framebuffer regions, controlled via addr_controller.

Purpose:

Stores per-frame pixel data for display output. Enables smooth animation, double buffering, and memory-efficient color representation using a small palette (16 colors).

Module: clk_wiz

Inputs:

clk_in1, reset

Outputs:

clk_out1 (25 MHz), clk_out2 (125 MHz), locked

Description:

The clk_wiz module is a clock wizard IP core generated in Vivado to provide multiple derived clock signals from a single input clock. In this design, it takes the main 100 MHz system clock and produces two separate output clocks: 25 MHz for the VGA display pipeline and 125 MHz for the HDMI serializer. It also provides a locked signal to indicate when the output clocks are stable and ready for use.

Purpose:

To generate the necessary pixel and high-speed clocks required for VGA and HDMI operation from the single system clock. This module ensures proper timing for video signal synchronization and transmission and is essential for coordinating frame updates and HDMI encoding across the system.

Module: palette_rom

Inputs:

- palette[3:0]: 4-bit color index selecting a color from the ROM

Outputs:

- red[7:0]: 8-bit red channel value
- green[7:0]: 8-bit green channel value
- blue[7:0]: 8-bit blue channel value

Description:

The palette_rom module implements a simple 16-entry color lookup table, where each entry maps a 4-bit color index to a 24-bit RGB color. It provides a direct mapping from palette codes—used in frame buffers or texture indices—to full 8-bit color channels for video output. The palette closely mirrors the standard IBM PC CGA/EGA 16-color scheme, making it compatible with retro-style graphics rendering.

The internal ROM is initialized using an initial block with hardcoded RGB values. Each color is packed as a 24-bit vector and decoded into red, green, and blue channels using a single assignment statement. The design is purely combinational, ensuring zero clock latency and allowing fast color resolution during rasterization or scan-out.

Purpose:

This module serves as a compact color decoder for graphics pipelines. It enables a memory-efficient way to represent colors in buffers using only 4 bits while still supporting 24-bit RGB output. It is ideal for use in FPGA-based graphics systems like VGA/HDMI text and pixel renderers, and is extensible to larger palettes or dynamic updates if needed.

Module: vga_controller

Inputs:

pixel_clk, reset

Outputs:

hs, vs, active_nblank, sync, drawX[9:0], drawY[9:0]

Description:

The vga_controller module generates the necessary timing signals for a 640×480 VGA display operating at 25 MHz pixel clock. It produces horizontal and vertical sync pulses (hs, vs), blanking signals (active_nblank), and real-time pixel coordinates (drawX, drawY) for each frame. Internally, it uses horizontal and vertical counters to track position and determine when to assert sync signals and enable pixel rendering.

Purpose:

To provide accurate synchronization and pixel coordinate generation for the VGA pipeline, enabling modules such as color_mapper and ball to render visual content on the screen in sync with the VGA/HDMI display timing standard. This module ensures that pixel updates occur only during the active display region and not during the blanking intervals.

Module: vga_to_hdmi

Inputs:

pix_clk, pix_clkx5, pix_clk_locked, rst,
red[3:0], green[3:0], blue[3:0],
hsync, vsync, vde,
aux0_din[3:0], aux1_din[3:0], aux2_din[3:0], ade

Outputs:

TMDS_CLK_P, TMDS_CLK_N,
TMDS_DATA_P[2:0], TMDS_DATA_N[2:0]

Description:

The vga_to_hdmi module instantiates a RealDigital IP core that converts standard VGA signals into HDMI-compliant TMDS (Transition Minimized Differential Signaling) outputs. It receives pixel clock and 5× pixel clock inputs, along with RGB color, sync, and blanking signals, and produces differential HDMI signals for output to an external monitor. Auxiliary and audio data inputs are unused in this lab.

Purpose:

To enable HDMI display output from an FPGA by translating VGA-style timing and color signals into HDMI format. This module bridges the gap between traditional VGA rendering logic and modern digital displays, allowing the system to visualize graphical output from modules like ball and color_mapper on a standard HDMI screen.

Module: texture_rom

Inputs:

- U[4:0], V[4:0]: Texture coordinate inputs (horizontal and vertical indices)

Outputs:

- out[3:0]: 4-bit color index corresponding to the (U, V) coordinate

Description:

The texture_rom module provides a compact, parameter-free interface for retrieving color indices from a synthetic or pre-defined texture pattern. It maps 5-bit U and V coordinates to a

4-bit output representing the color index, which can then be interpreted by a palette or used directly for pixel color generation in graphics pipelines.

The current implementation is a placeholder logic (out = U >> 1), which creates a repeating horizontal stripe pattern across the U-axis. However, this module can be easily extended to support more complex procedural textures, look-up tables from ROMs, or embedded image patterns. As such, it is a versatile plug-in component for rendering systems where visual textures, tiles, or backgrounds are needed.

Purpose:

This module is designed to provide texture sampling functionality in an HDL-based graphics system. It abstracts the process of retrieving pixel color data based on 2D coordinates and is ideal for use in GPU-style rasterization pipelines, educational VGA/HDMI projects, or games on FPGA. Its small footprint and simple interface make it easily replaceable with more sophisticated texture ROMs or BRAM-backed image data sources.

Module: hdmi_text_controller_v1_0

Inputs:

- AXI4-Lite interface clock and reset
  axi_aclk, axi_aresetn
- AXI4-Lite write channel
  axi_awaddr, axi_awprot, axi_awvalid, axi_wdata, axi_wstrb, axi_wvalid, axi_bready
- AXI4-Lite read channel
  axi_araddr, axi_arprot, axi_arvalid, axi_rready
- GPU status signals
  gpu_ready[1:0]

Outputs:

- AXI4-Lite response signals
  axi_awready, axi_wready, axi_bresp, axi_bvalid, axi_arready, axi_rdata, axi_rresp, axi_rvalid
- Control signals to GPU
  line_data[63:0], control[3:0]

Description:

The hdmi_text_controller_v1_0 module is a top-level wrapper for the HDMI text rendering system used in ECE 385. It instantiates the internal AXI4-Lite peripheral hdmi_text_controller_v1_0_AXI, which handles memory-mapped communication with a host processor (e.g. MicroBlaze). This design enables the processor to update text line data and control signals for rendering on an HDMI display.

The top-level module serves as a structural container where additional submodules such as a clocking wizard, VGA synchronization generator, font ROM, character mapper, and HDMI encoder will be instantiated. These components form a complete pipeline from AXI-driven memory-mapped updates to pixel-level video generation. The HDMI video stream can initially display a basic pattern (such as a blue screen or gradient) to confirm signal correctness before text features are fully implemented.

Purpose:

This module forms the backbone of a hardware/software co-designed text rendering system targeting HDMI output. By delegating AXI communication to a dedicated submodule, it simplifies integration with display control logic and encourages modular development. The design supports flexible educational extensions such as animation, color customization, and system-level debugging via memory-mapped I/O. It is particularly useful for demonstrating embedded graphics pipelines in FPGA-based systems.

Module: hdmi_text_controller_v1_0_AXI

Inputs:

- Clock and reset signals for the AXI4-Lite interface
  S_AXI_ACLK, S_AXI_ARESETN
- AXI4-Lite write channel signals
  S_AXI_AWADDR, S_AXI_AWPROT, S_AXI_AWVALID, S_AXI_WDATA, S_AXI_WSTRB, S_AXI_WVALID, S_AXI_BREADY
- AXI4-Lite read channel signals
  S_AXI_ARADDR, S_AXI_ARPROT, S_AXI_ARVALID, S_AXI_RREADY
- Status inputs from GPU logic
  gpu_ready[1:0]

Outputs:

- AXI4-Lite response signals
  S_AXI_AWREADY, S_AXI_WREADY, S_AXI_BRESP, S_AXI_BVALID, S_AXI_ARREADY, S_AXI_RDATA, S_AXI_RRESP, S_AXI_RVALID
- Control and data outputs to GPU
  line_data[63:0], control[3:0]

Description:

The hdmi_text_controller_v1_0_AXI module implements the AXI4-Lite memory-mapped interface for a text-based HDMI controller system. It allows a processor (e.g., MicroBlaze) to configure text rendering parameters by writing to a small set of memory-mapped registers. The module decodes read and write transactions from the AXI interface and stores relevant data in internal slave registers. These registers include character data (line_data), text

attributes (control), and status flags indicating synchronization with GPU rendering cycles.

The design supports fine-grained control using byte strobes and handles proper handshaking for read and write operations, conforming to the AXI4-Lite protocol. Special attention is given to the gpu_ready handshake signals, which govern the synchronization between the CPU-controlled updates and GPU consumption of text data. The module raises or clears status bits automatically to reflect GPU processing stages, such as data readiness or processing completion.

Purpose:

This module serves as the memory-mapped command interface for the HDMI text controller system. It decouples the software logic (e.g., from a MicroBlaze or ARM processor) from the hardware text rendering pipeline, enabling flexible CPU-GPU communication through a standard AXI4-Lite interface. Designed for use in ECE 385, the module demonstrates how hardware and software components can be co-designed for custom graphics systems, focusing on register design, bus protocols, and synchronization techniques.

Software Main Program Description (main.c)

This program runs on the MicroBlaze soft-core processor within the FPGA and acts as the software driver for a custom HDMI graphics pipeline. Its main function is to render a rotating cube in 3D space by performing transformations and sending interpolated scanline data to hardware through an AXI4-Lite interface for HDMI video output.

Main Loop Functionality

The main loop continuously performs the following operations:

1. Listens for USB keyboard input. The keys W, A, S, D, Q, and E control rotation of the cube around the X, Y, and Z axes.
2. Maintains the cube's orientation using quaternions to allow smooth, gimbal-lock-free rotation.
3. Transforms the cube's 8 vertices from model space to world space, then projects them to screen space with associated depth values.
4. Iterates through the 12 triangles of the cube and calls rasterise_triangle() to interpolate texture coordinates and depth values for each scanline.
5. Sends each scanline to the hardware using push_scanline(), which packs the scanline's coordinates and attributes into two 32-bit words and writes them to the GPU via AXI-Lite.
6. After all scanlines of a frame are sent, the program calls gpu_frame_done() to signal hardware that the frame is complete and ready for display.

USB Keyboard Control

Keyboard input is handled via the MAX3421E USB controller. The following controls are supported:

- W / S: Rotate around the X axis
- A / D: Rotate around the Y axis
- Q / E: Rotate around the Z axis

Each keypress adjusts the rotation angles. A delta quaternion is calculated and applied to the current orientation, which is normalized and used to compute the updated rotation matrix.

GPU Communication

Each scanline is packed into two 32-bit words by push_scanline(), containing the Y coordinate, start and end X coordinates, U/V texture coordinates, and Z depth values. These are written into AXI-mapped registers on the FPGA to drive the rendering process. When the entire frame is processed, gpu_frame_done() is used to pulse a control register, notifying the hardware to begin displaying the rendered frame.

Model and Transform Support

The cube model consists of 8 vertices and 12 triangles. The software includes support for 3D vector and matrix operations, quaternion multiplication, normalization, and conversion to rotation matrices. These mathematical tools enable accurate and efficient real-time rotation and projection of the cube.

This program demonstrates a full hardware-software co-designed 3D rendering pipeline with real-time display output via HDMI, suitable for use in embedded graphics labs or educational FPGA projects.

Rasterizer Module Description (rasteriser.c)

This module implements a **software rasterizer** that processes one triangle at a time and converts it into horizontal scanlines, which are sent to the hardware GPU pipeline. The rasterizer works in screen space using projected vertex positions, interpolated texture coordinates (U, V), and depth (Z).

Functionality

The public function rasterise_triangle(int tri_index) takes an index into a predefined triangle list (g_tris[]) and performs triangle rasterization. Each triangle consists of three vertex indices and their associated texture coordinates.

Steps performed by the rasterizer:

1. **Vertex Fetch and Sorting**:
    - Retrieves the screen-space coordinates and depth values of the triangle's three vertices.

    - Vertices are sorted in ascending order by Y-coordinate to simplify scanline filling.

2. **Edge Slope Calculation**:
    - Computes the X, U, V, and Z slopes along the triangle's edges using linear interpolation.

3. **Top and Bottom Half Triangle Handling**:
    - The triangle is split into two halves: top and bottom.
    - For each horizontal scanline between the top and bottom Y-values, the interpolated start and end X positions are calculated, along with corresponding U, V, and Z values.

4. **Scanline Emission**:
    - For each valid scanline, the function emit_span() is called to interpolate attributes across the horizontal line and pass them to scanline_out().
    - Clipping is applied to ensure that the output X range lies within the screen (0 to 319).
    - Depth and texture values are clamped and quantized to match 7-bit Z and 5-bit U/V precision.

5. **Output Format**:
    - The scanline_out() function is expected to forward data to the hardware using a memory-mapped AXI4-Lite interface.

Triangle Format

Each triangle in g_tris[] is defined by:

- v[3]: indices of the three vertices in the global cube model
- u[3], vtex[3]: corresponding U and V texture coordinates (range 0–31)

Purpose

This module bridges software-based geometry processing and hardware-accelerated rendering. It allows complex 3D objects like cubes to be rasterized into scanlines on the CPU, while the FPGA handles interpolation, Z-buffering, and final pixel output to HDMI. It demonstrates core concepts of triangle rasterization, including edge walking, barycentric interpolation, and per-pixel attribute generation.

GPU AXI-Lite Interface Driver (gpu_axi_lite.c)

This module implements the low-level communication interface between the MicroBlaze processor and the custom GPU hardware module using AXI4-Lite memory-mapped registers.

It provides essential control functions to send scanline data and synchronize frame rendering.

Function Descriptions

void gpu_init(void)
Initializes the GPU interface. Currently a placeholder with no setup logic, as no initialization is required for the memory-mapped AXI registers.

void gpu_wait_ready(void)
Blocks until the GPU is ready to accept new data.
It repeatedly reads from the GPU_DATA_CTRL register and waits until the DONE bit is set, indicating that the GPU has completed processing the previous command.

void gpu_send_scanline(uint32_t lo, uint32_t hi_data)
Sends one scanline of data to the GPU hardware.

- Waits for the GPU to become ready.
- Writes the high 32-bit data word (containing X0, X1, Z0, Z1).
- Writes the low 32-bit word (containing Y, U0, U1, V0, V1).
- Issues a RUN command to signal the GPU to start processing the scanline.

void gpu_frame_done(void)
Notifies the GPU that all scanlines for the current frame have been sent.

- Waits for the GPU to finish its current operation.
- Writes the FRAME_DONE command to the control register.
- Optionally prints a visual delimiter in the console to indicate the end of a frame.

Register Summary

Register	Description
GPU_DATA_CTRL	Control register for command signaling
GPU_DATA_HI	High data word: X0, X1, Z0, Z1
GPU_DATA_LO	Low data word: Y, U0, U1, V0, V1, etc.

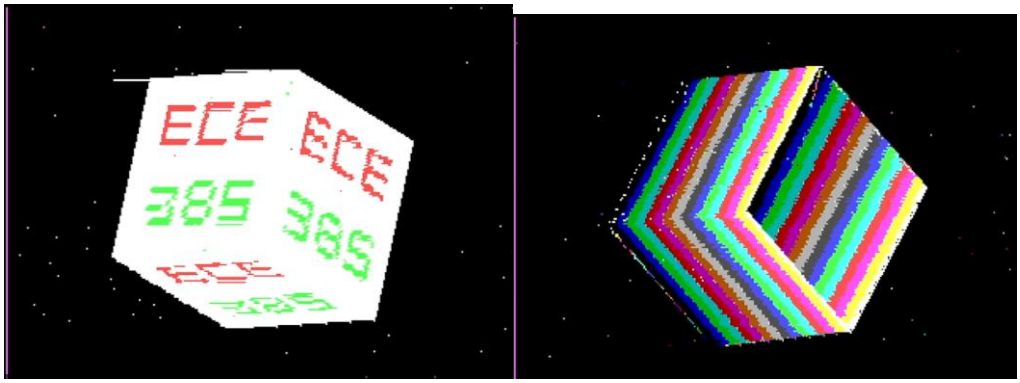
Purpose

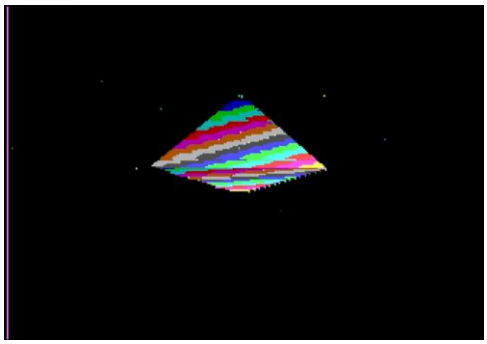
This module abstracts the hardware-level AXI-Lite protocol and provides a minimal but effective software interface for GPU command issuance. It is used in tandem with the software rasterizer to stream triangle scanlines and manage frame synchronization, forming the software control core of a hardware-accelerated 3D graphics pipeline.

Simulation

There is no simulation used in this project

Images





Design Utilization

Design Statistic Table

LUT	3696
DSP	3
Memory	68
Flip-Flop	2666
Latches	0
Frequency	20.63Mhz
Static Power	0.078w
Dynamic Power	0.373w
Total Power	0.451w

Problems Encountered

At the beginning of the project, the initial design intended to implement full hardware-based

rasterization by instantiating twelve parallel computational units, each responsible for determining whether the current pixel fell within one of the model's triangles. However, this approach quickly ran into FPGA resource limitations, making it impractical to fit all computation units within the available logic fabric. To resolve this, a MicroBlaze soft processor was introduced to offload the vertex processing and triangle interpolation tasks to software, significantly reducing hardware resource usage while maintaining flexibility.

Another issue encountered during development involved partial or missing rendering of the 3D object. Certain regions of the model would consistently fail to appear on screen, and this behavior was reproducible under the same camera angles or frame states. Initially, this was suspected to be a calculation error within the MicroBlaze software, such as incorrect vertex transformations or scanline generation. However, further debugging revealed that the actual problem stemmed from an unreliable handshaking mechanism between the AXI4-Lite interface and the GPU logic. Specifically, scanlines were being sent before the GPU was fully ready to process them, resulting in lost or incomplete data. By introducing additional handshake signals and ensuring proper synchronization, this issue was resolved, and full-frame rendering was achieved reliably.

A final unresolved issue remains: under specific angles or orientations, portions of the rendered object appear to be clipped or missing. This may be related to limitations in the projection logic or screen-space clipping boundaries, and further investigation is required to determine whether this is a geometric issue or a flaw in the rasterization process.

Conclusion

This project successfully demonstrates a functional 3D rendering pipeline on an FPGA, capable of drawing arbitrary triangle-based models in real time with HDMI output. By combining a software-controlled MicroBlaze processor with a custom hardware rasterization engine, the system achieves efficient division of labor between geometry processing and pixel-level rendering.

Initially planned as a fully hardware-accelerated design, the project was adapted to include MicroBlaze after hitting resource constraints, allowing the system to scale beyond a fixed cube model. Software-generated scanlines are passed to the GPU via AXI4-Lite, where depth testing and framebuffer updates are performed in hardware. With dual-port BRAM and double buffering, the output remains smooth and tear-free.

Through this process, various engineering challenges were encountered—including resource limitations, synchronization bugs, and rendering glitches—and were addressed through iterative debugging and design refinement. The final system illustrates key concepts in hardware/software co-design, 3D graphics rendering, and embedded architecture, laying a solid foundation for future improvements such as perspective projection, lighting, and model loading.